



# Distribuirani sistemi

---

Distribuirani fajl sistemi

# Fajl sistem

## \* fajl

- imenovani skup podataka trajno zapamćen na disku (ili nekom drugom medijumu, CD, DVD,..)

## \* Direktorijumi

- Specijalna vrsta fajlova koja služi za organizaciju drugih fajlova
- Sadržaj direktorijuma može ukazivati na druge direktorijume ili fajlove

## \* Fajl sistem je komponenta operativnog sistema odgovorana za organizaciju, skladištenje, imenovanje, pretraživanje, deljenje i zaštitu fajlova.

- Obezbeđuje korisnički interfejs za rad sa fajlovima.
  - operacije nad fajlovima
    - Open, Close, Read, Write, Seek,
  - Operacije nad direktorijumima
    - Lookup, List, Add, Remove, Rename, ...

# smeštanje fajlova na disku

## \* kontinualno

- blokovi fajla su zapamćeni u sukcesivnim sektorima na disku
  - Direktorijum ukazuje na lokaciju prvog bloka

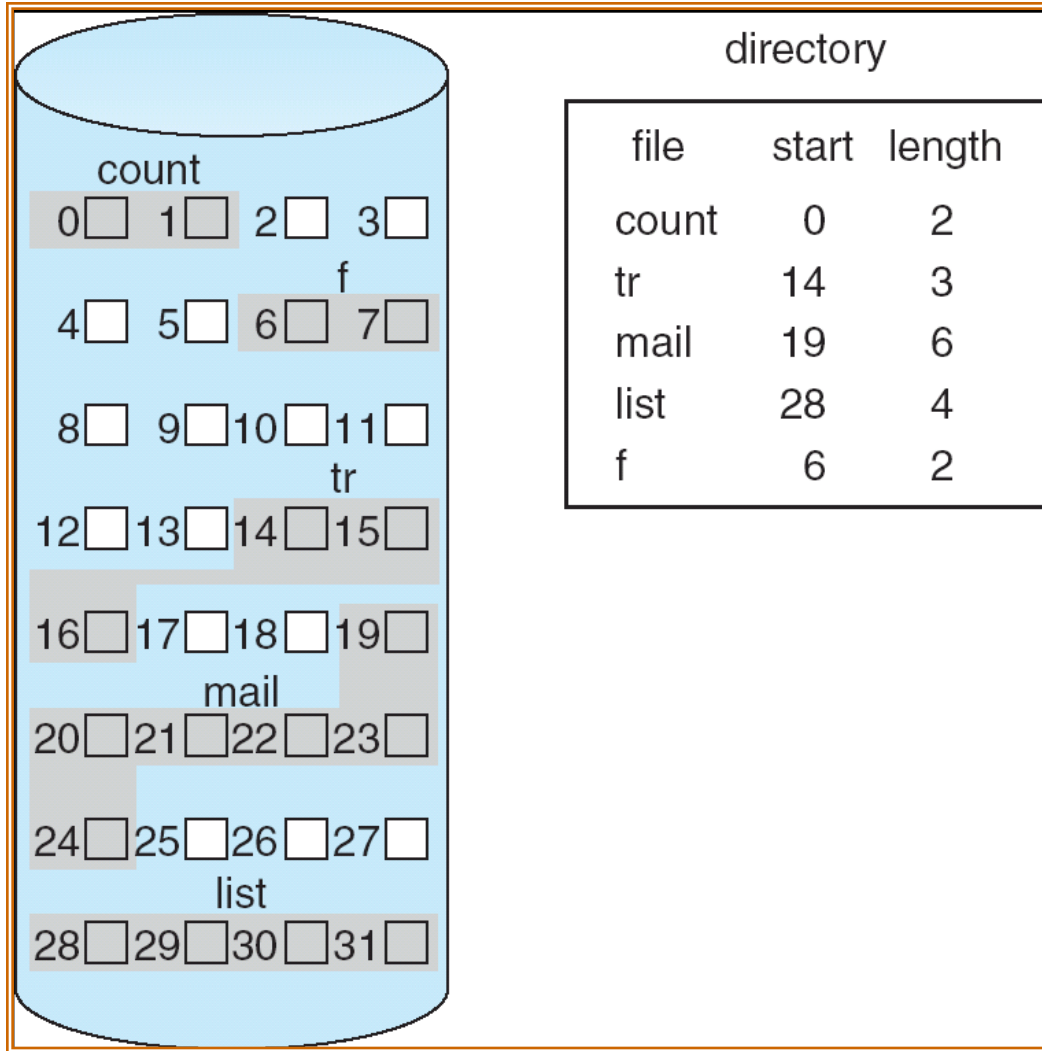
## \* ulančano

- blokovi jednog fajla mogu biti rasejani na disku, kao lančane liste
  - Direktorijum ukazuje na lokaciju prvog bloka

## \* Indeksirano

- poseban blok (indeks blok) sadrži pointere na blokove fajla
  - direktorijum ukazuje na lokaciju indeks bloka

# kontinualno smeštanje

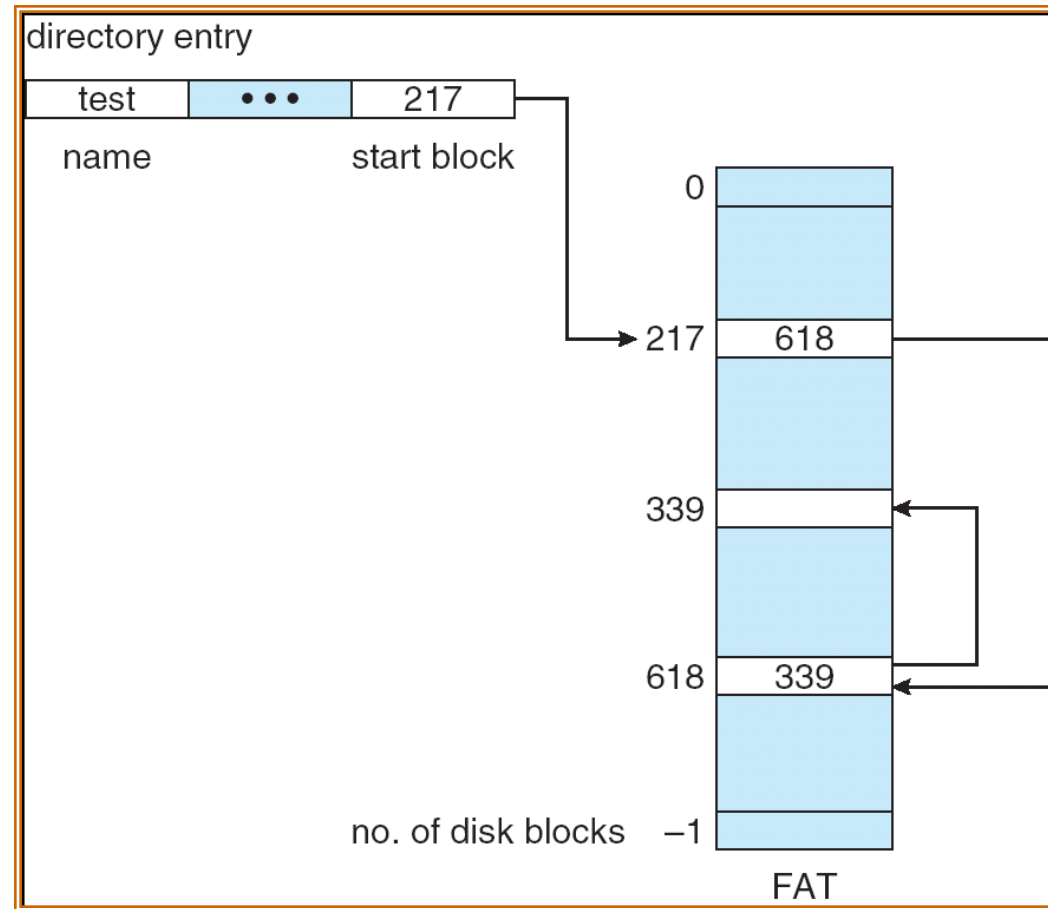


idealno za velike, statične  
fajlove (baze podataka, OS  
kod, Multi-media video i  
audio)

- CD-ROM, DVD

# ulančano smeštanje

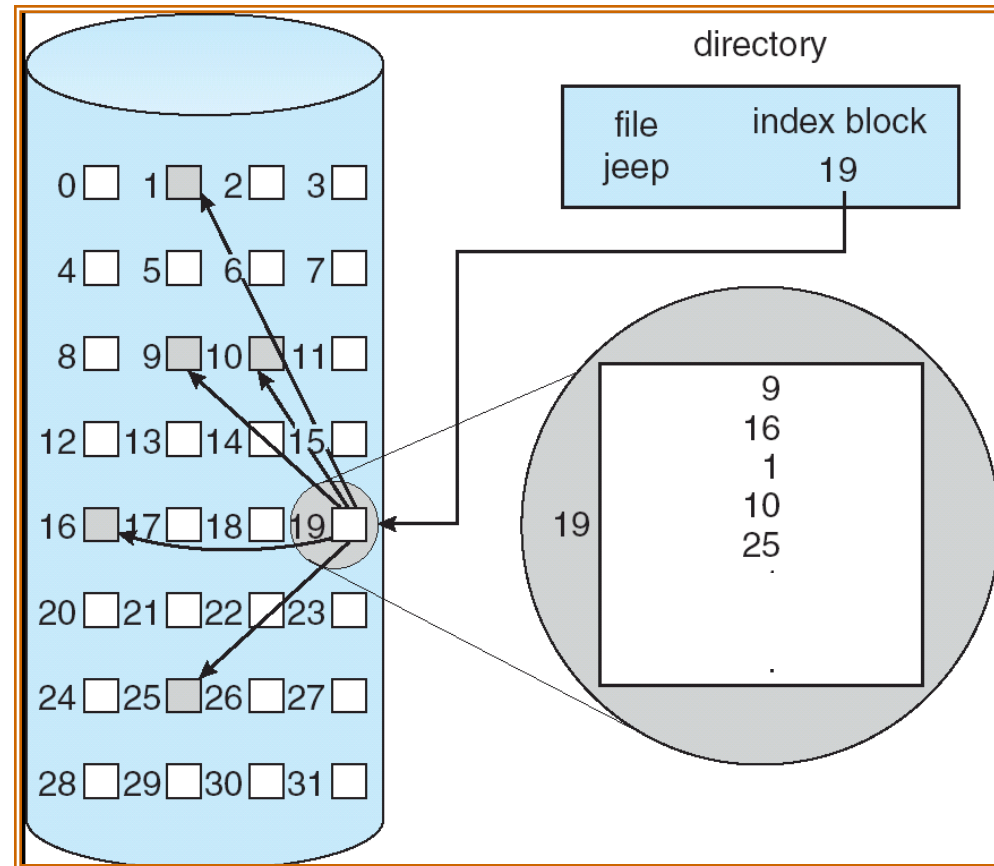
- Direktorijum ukazuje na prvi blok fajla
- svaki blok ukazuje na sledeći blok (ili *EOF*)



# indeksirana alokacija

- direktorijum ukazuje na indeks blok *i-node*:

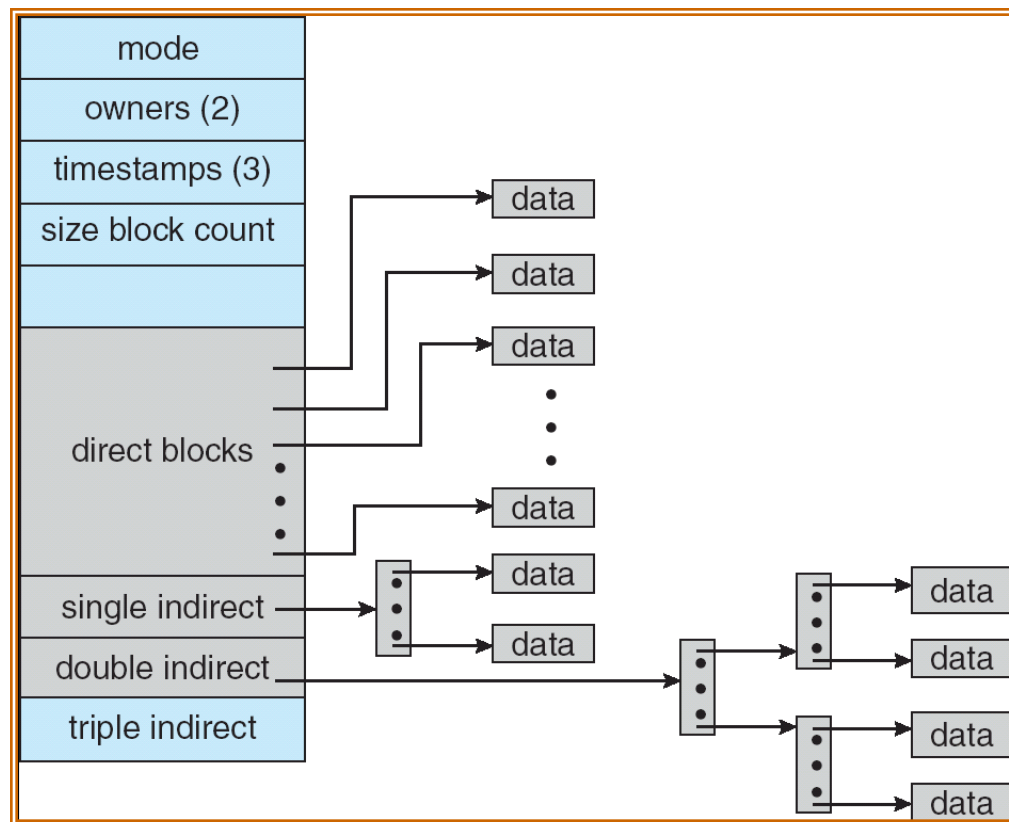
- i-node sadrži adresu svakog bloka fajla
- svakom bloku se pristupa preko i-node
- za svaki pristup fajlu zahteva se pristup i-node



# izgled i-node u UNIX/LINUX

\* Referenciranje fajlu se ne vrši na osnovu imena već na osnovu i-node

- Svaki direktorijum sadrži tabelu sa imenom fajla i brojem inode. (može se izlistati sa `ls -li`)
  - Svi inode su zapamćeni u spelicajalnoj lokaciji na disku – super blok
  - Inode pamti attribute fajla i pointere na blokove u kojima su zapamćeni podaci datog fajla (12 direktnih pointera, 1 indirektni, 1 dvostruki indirektni i 1 trostruki indirektni pointer).



# Distribuirani fajl sistem(DFS)

- \* Implementacija DFS zahteva sve komponente tradicionalnog fajl sistema sa dodatnim komponentama za klijent-server komunikaciju, imenovanje i lociranje fajlova u distribuiranom sistemu.
- \* DFS -Fajl sistem se nalazi na više autonomnih mašina.
  - omogućava da se pristup udaljenim fajlovima obavlja isto kao lokalnim fajlovima.
- \* DFS su bazirani na klijent-server modelu.
  - DFS pruža API za pristup fajl sistemu
    - API se razlikuju za različite DFS
- \* DFS se sastoji od
  - direktorijumskog servisa
    - bavi se imenovanjem i lociranjem fajlova
    - vrši preslikavanje tekstualnog imena fajla u jedinstveni fajl identifikator na osnovu kojeg se može locirati fajl u DS
    - imena moraju da budu jedinstvena u celom DS, a ne unutar jednog fajl sistema
    - direktorijumski servis može biti na odvojenoj mašini (sadrži i-node)
      - u tradicionalnim fajl sistemima to nije slučaj (direktorijumski servis i fajl servis su na istoj mašini)
  - fajl servisa
    - bavi se održavanjem sadržaja fajla
    - može da se uradi lokalno u svakom serveru u kome se fajl nalazi

## \* DFS treba da obezbedi

- transparentnost pristupa.

- Udaljenim i lokalnim fajlovima se pristupa na isti način

- Lokaciona transparentnost

- Isti način imenovanja se koristi i za lokalne i za udaljene fajlove; ime fajla ne otkriva njegovu fizičku lokaciju

- Transparentnost konkurencije

- Svi klijenti imaju isti pogled na stanje sistema; ako neki proces modifikuje fajl, procesi na istom ili udaljenom sistemu će videti modifikaciju na koherentan način.

- Transparentnost replikacije

- Fajlovi mogu biti replicirani na više servera; klijent nije svestan replikacije

- Migraciona transparentnost

- Fajl može da migrira sa jednog servera na drugi a da to ne utiče na pristup fajlu od strane klijenta

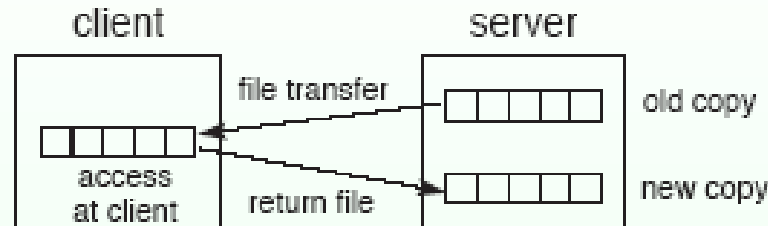
- Heterogenost

- Pristup udaljenim fajlovima je moguć bez obzira na razlike u hardveru i OS.

# Modeli pristupa udaljenom fajlu

## \* Upload/download model

- Jedini servisi koji su obezbeđeni su read i write.
  - read preuzima (download) fajl sa servera i pamti ga na klijent mašini; obavlja operacije nad fajlom lokalno; kada završi vraća (upload) fajl na server.
  - Primer Internetov FTP servis
- Jednostavan model
- performanse su bolje jer se sve obavlja lokalno, nema mrežnog saobraćaja
- Problemi:
  - šta ako klijent nema dovoljno prostora da zapamti ceo fajl?
  - Šta ako je klijentu potreban samo mali deo fajla?
  - Šta ako i drugi klijenti žele da modifikuju fajl?



upload/download model

# modeli pristupa udaljenom fajlu (nast.)

## \* Model udaljenog pristupa

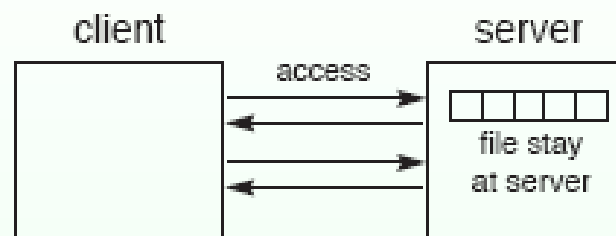
- Fajl servis obezbeđuje izvršenje operacija na udaljenoj mašini (open, close, read, write, read byte, write byte,...); operacije se izvršavaju na serveru (fajl se ne pomera sa servera)
  - npr. klijent može zahtevati da se otvori određeni fajl, a zatim da se pročita određeni blok fajla i samo taj blok će biti prenet klijentu
  - sve se realizuje preko RPC mehanizma

## \* Prednost

- lakše je implementirati deljenje fajlova
  - ako neki klijent modifikuje fajl ta modifikacija je odmah vidljiva svim klijentima

## \* Nedostaci

- Moguće zagušenje mreže i servera
  - Pristup serveru traje za vreme celog pristupa fajlu umesto samo za vreme upload/download.
- lošije performanse
  - čitanje sa udaljenog diska je sporije nego čitanje sa lokalnog



remote access model

# Statefull i stateless server

## \* Stateless server

- server ne pamti koji klijenti pristupaju nekom fajlu, koji fajlovi su otvoreni
- nikakva informacija o klijentskim zahtevima se ne pamti na serveru
- sve informacije potrebne da bi se ob služio zahtev mora da pamti i obezbedi klijent pri svakom zahtevu
- otporniji je na otkaz servera jer se nikakve informacije ne pamte na serveru
  - ako server otkaže i ponovo se podigne nikakve informacije neće biti izgubljene
- ako neki klijent modifikuje fajl, server nema načina da obavesti ostale klijente da je fajl modifikovan
  - klijent mora da vodi računa o konzistenciji, jer server nema nikakve informacije

## \* Stateful server

- server pamti informacije o tome koji klijent je otvorio koji fajl
- kraće su poruke u kojima klijent šalje zahteve serveru
- Bolje performanse
- lakše je ostvariti konzistenciju
  - ako jedan klijent modifikuje fajl, server zna koji klijenti pristupaju tom fajlu pa može preduzeti akcije da obavesti druge klijente da je fajl promenjen
  - moguće primeniti fajl locking (zaključavanje)
- teže postići tolerantnost na otkaze
  - ako server otkaže, pa se ponovo podigne informacije o tome koji klijenti pristupaju kom fajlu će biti izgubljene
  - Mora se periodično pamtit stanje servera da bi se izvršio oporavak od greške
    - Beleženje stanja može biti dosta zahtevno sa stanovišta performansi

# Projektantska pitanje

- \* Imenovanje: lociranje fajla/direktorijuma u DFS na osnovu imena.
- \* Semantika deljenja fajlova
- \* Lokacija keša: disk, memorija, oba.
  - Politika upisa: ažuriranje izvora kada se keš modifikuje.

# Imenovanje fajlova

## \* Pitanja

- – kako su fajlovi imenovani?
- – Da li ime fajla govori o njegovoj lokaciji u DS?
- – Da li se ime fajla menja ako fajl promeni lokaciju?
- – Da li se ime fajla menja ako korisnik (klijent) promeni lokaciju?

## \* Lokaciona transparentnost:

- ime fajla ne otkriva fizičku lokaciju fajla.

## \* transparentnost migracije (lokaciona nezavisnost ):

- Ime fajla ne mora da se promeni ako se promeni njegova lokacija.

## \* Većina šema za imenovanje fajlova nema migracionu transparentnost, ali može imati lokacionu transparentnost.

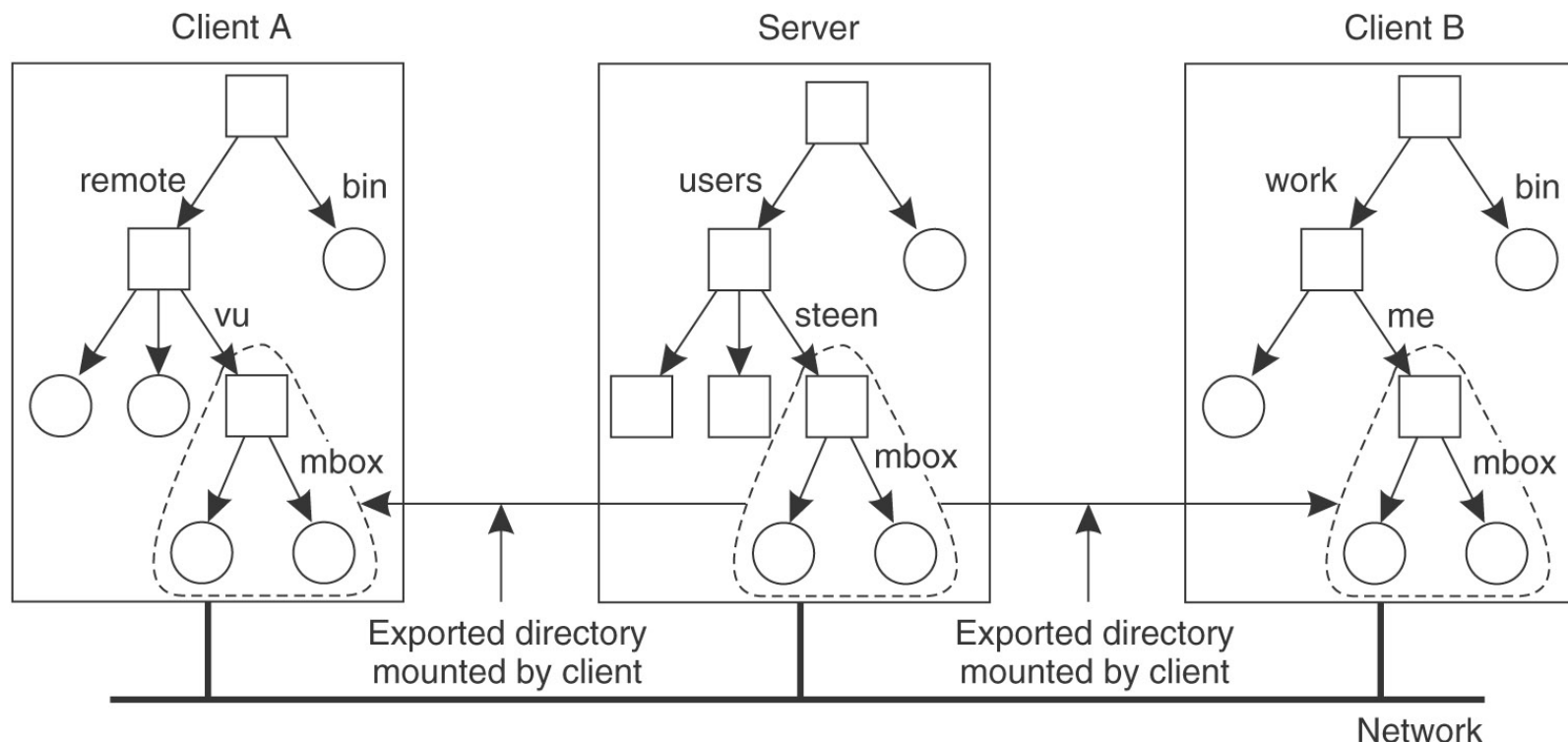
# Tri načina imenovanja

- \* Imenovanje fajlova kombinacijom imena hosta i lokalnog imena fajla. (ime\_hosta:lokalno\_ime)
  - Garantuje jedinstvenost imena.
    - Npr. Windows Network Places
    - URL kod http
      - [http://match.pmf.kg.ac.rs/electronic\\_versions/Match81/n1/match81n1\\_163-174.pdf](http://match.pmf.kg.ac.rs/electronic_versions/Match81/n1/match81n1_163-174.pdf)
  - Prednosti:
    - Pronalaženje fajla je jednostavno
  - Nedostaci:
    - Korisnik mora znati kompletno ime i svestan je koji su fajlovi lokalni a koji udaljeni.
    - Ime fajla je lokaciono zavisno i fajl se ne može pomerati.
    - Nema transparentnosti. Pomeranje fajla na drugi host, zahteva promenu imena.
- \* Mounting: montiranje udaljenog direktorijuma na lokalni direktorijum. Lokaciona transparentnost se postiže nakon montiranja. (koristi se u Sun NFS).
  - Montiranim direktorijumima se pristupa kao lokalnim.
    - npr. mapirani drajv u Windowsu

# Mounting

- \* Ubacivanje (mounting) udaljenog direktorijuma na lokalni direktorijum. (NFS – Sun-ov Network File System)
- \* Svaki host ima skup lokalnih imena za obraćanje udaljenim lokacijama
  - Ova imena se zovu tačke montiranja jer su udaljeni fajlovi ili direktorijumi povezani i prikačeni na lokalni fajl sistem.
- \* Svaki host ima tabelu (/etc/fstab) koja definiše preslikavanje
  - <remote path name@ machine name> u <local path name>.
- \* Kada se sistem uključi, ostvaruje se povezivanje sa udaljenim serverom.
- \* Nakon toga korisnik pristupa udaljenom fajlu korišćenjem lokalnog imena, a DFS vodi računa o preslikavanju
- \* • Prednost: lokaciona transparentnost – kada se ostvari povezivanje sa udaljenim serverom klijent ne mora da vodi računa da li je fajl kome pristupa lokalni ili udaljeni
- \* Udaljeni server može da se promeni a da to ne utiče na imenovanje fajla
- \* • Nedostatak: isti fajl može imati više imena u zavisnosti od klijenta

# Mountovanje udaljenih direktorijuma na lokalni fajl sistem



## \* Imena nisu globalno jedinstvena

### • Isti fajl se različito vidi

- Server vidi fajl mbox : */users/steen/mbox*
- Client A vidi fajl mbox : */remote/vu/mbox*
- Client B vidi fajl mbox : */work/me/mbox*



# Tri načina imenovanja (nast.)

- ✱ Totalna integracija komponenti fajl sistema - svi fajlovi u sistemu pripadaju jedinstvenom prostoru imena.
  - Imena su jedinstvena u celom sistemu
  - Ime fajla je isto bez obzira sa kog čvora se pristupa fajlu
    - Ovo može da funkcioniše samo između sistema koji čvrsto sarađuju. (Berkeley Sprite, CMU Andrew)

# Semantika deljenja fajlova

## \* Fajl sistem deli mnogo korisnika

- Šta se dešava ako dva procesa pristupaju istom fajlu?

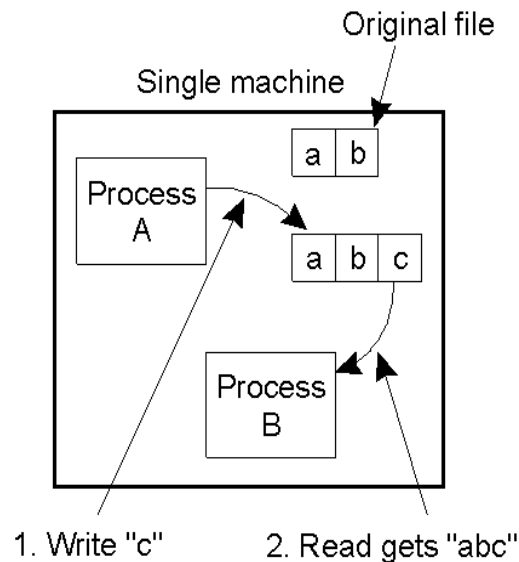
## \* Sekvencijalna semantika (unix semantika)

- Apsolutno vremensko uređenje svih operacija.
- Svaka read operacija nad fajlom vidi efekte prethodnih write operacija.
- Sekvencijalnu semantiku u DFS je lako ostvariti ako svakim fajlom upravlja jedan server i klijenti ne keširaju fajlove.
  - Tada redosled read i write pristupa određuje server.
- Ako se ne koristi keširanje na strani klijenta, javlja se problem sa performansama – klijenti će se obraćati serveru za svaku operaciju na fajlu (čak i čitanje jednog bajta)
- Problem performansi se može ublažiti ako se dozvoli keširanje na strani klijenta.
  - Javlja se problem konzistencije podataka: Ako klijent modifikuje podatak u kešu, a drugi klijent pročita podatke sa servera, dobiće ustajale podatke (sekvencijalna semantika više ne važi!)
  - Rešenje: koristiti write-through politiku upisa.
    - Ne rešava problem ustajalih kopija u keševima drugih klijenata – server mora da obavesti ostale klijente da imaju ustajale podatke.
  - Drugo rešenje – ublažiti semantiku

# Semantika deljenja fajlova

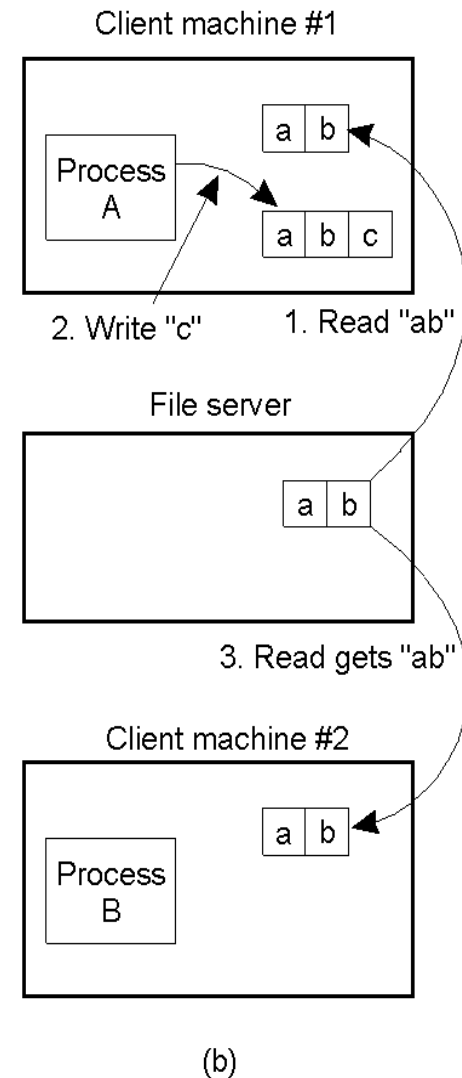
## \* Semantika sesije

- Sesija – serija pristupa fajlu između otvaranja i zatvaranja fajla.
- Promene učinjene na fajlu su vidljive samo klijent procesu (moguće i drugim procesima na istoj mašini)
- Drugim procesima su promene vidljive nakon zatvaranja i ponovnog otvaranja fajla.



(a)

a) Sekvencijalna semantika



(b)

b) semantika sesije

# Keširanje

## \* Keširanje u memoriji servera

- Jednostavno i transparentno za korisnika
- Sve akcije nad fajlovima obavlja server
- Mrežni sobračaj nije smanjen
- Server je potencijalno usko grlo

## \* Keširanje fajlova na strani klijenta

- Na klijentovom disku
  - Dovoljno prostora za pamćenje velikih fajlova, pouzdano
  - Brži pristup fajlu nego kroz mrežu
  - Server se samo povremeno kontaktira (rasterećen je, sistem je skalabilan)
- U klijentovom kešu
  - Uobičajeno rešenje
  - Brzi pristup podacima iz fajla
  - Nepouzdanost – u slučaju otkaza klijenta podaci mogu biti izgubljeni
- Sa keširanjem na strani klijenta javlja se problem keš konzistencije!

# Politike upisa u keš

## \* Kada modifikovani sadržaj keša treba da se prenese na server?

### ● *Write-through* politika

- Zajedno sa modifikacijom keša modifikuje se i fajl na serveru.
- Prednost: pouzdanost, otkaz klijenta ne znači gubitak podataka.
- Nedostatak : upisi na server se obavljaju svaki put kada se modifikuje keš.
- Prednost keširanja se koristi samo u slučaju read operacije.

### ● Politika zakašnjenog upisa

- Upis na server se obavlja nakon određenog vremena (npr. na svakih 30sec)
- Prednost: male i česte promene ne povećavaju mrežni saobraćaj.
- Nedostatak : manje pouzdano, osetljivo na otkaz klijenta

### ● Upis u trenutku zatavaranja fajla.

- Redukuje se broj upisa na disk
- Nepouzdanost – ako klijent otkaže podaci će biti izgubljeni

# Provera konzistentnosti podataka

✱ Da li su podaci u lokalnom kešu konzistentni sa master kopijom (serverskom kopijom)?

- Provera inicirana od strane klijenta

- Klijent kontaktira server da bi utvrdio da li je njegova kopija konzistentna sa serverskom kopijom
- Provera se obavlja kod svakog pristupa fajlu ili samo prilikom otvaranja fajla
- Česta provera validnosti podataka u kešu može da anulira prednosti keširanja

- *Server-inicirana provera*

- Server beleži prisustvo fajlova u keševima klijenata
- Kada server detektuje potencijalnu nekonzistentnost, reaguje.